

CONTENTS

- Overview
- 1 Why you can't just point a model at a PDF
- 2 Extracting structured context from raw documents
- 3 Generating synthetic data from seed examples
- 4 Validating synthetic data before you trust it
- 5 Why control over the data matters
- Glossary
- Check yourself
- Where to go next

Synthetic Data Generation for Smarter AI Workflows

Turn a handful of real examples into a large, validated training set without hand-labeling everything yourself.

For developers who want to fine-tune or ground a model on their own documents and are short on labeled data · 27 July 2026

[Watch the source video](#)[Read the condensed version](#)[Read the highlights](#)

BEFORE YOU START

- Basic familiarity with how language models are fine-tuned on question-and-answer pairs
- General idea of what retrieval-augmented generation (RAG) or document-grounded chatbots are

Overview

Suppose you want a chatbot that can answer questions from a specific research paper. A base language model was not trained on that paper, so it cannot reliably answer questions about it. Fine-tuning could fix this, but fine-tuning needs training examples, and a single paper does not contain enough of them. This is the data problem behind almost every domain-specific AI project: the raw source material exists, but it is not in a form a model can learn from, and there usually is not much of it.

The fix is a three-stage pipeline. First, extract structured content from unstructured documents. Second, turn a small amount of that structured content into a handful of hand-written question-and-answer examples, called seed data. Third, use a model to expand that seed data into a much larger set of new, similar examples: synthetic data. The word "synthetic" does not mean fake or low-quality here — it means generated rather than manually written, while still being checked for accuracy against the source.

This overview walks through each stage, the open source tools that implement them (Docling for extraction, SDGHub for generation), the validation checks that keep synthetic data trustworthy, and the practical reasons teams reach for this approach: privacy, class balance, and reproducible pipelines.

KEY TAKEAWAYS

- ✓ Models cannot learn directly from raw documents; unstructured text, tables, and equations must first be converted into structured data.
- ✓ A small set of hand-written question-and-answer pairs, called seed data, teaches a generation model the pattern and style to imitate.
- ✓ Synthetic data generation is a pipeline, not a single API call: it generates candidate examples, transforms them, and validates them before they are used.
- ✓ Validation checks synthetic examples for faithfulness to the source, relevance to the question, and diversity across the dataset.
- ✓ Generation pipelines can run entirely inside your own environment, connecting to local or self-hosted models, so source data never has to leave it.
- ✓ Synthetic data lets you preserve privacy, balance rare classes, and augment thin domains without collecting more real-world data.
- ✓ Because the pipeline is generate-validate-deploy on repeatable steps, the resulting dataset is reproducible — a requirement for most enterprise AI workflows.

01 Why you can't just point a model at a PDF

0:00

A model can't learn from a document directly, and even structured content usually isn't enough of it to shift a base model's weights.

Imagine you found a paper on quantum batteries and want to ask it questions directly, like how the batteries store energy. A base language model was not trained on that specific paper, so unless it retrieves the text at query time, it cannot answer accurately. Fine-tuning is one way to bake that knowledge into the model's weights, but fine-tuning is a supervised learning problem: it needs input-output examples, not a wall of prose.

A scientific paper is unstructured from a model's point of view: paragraphs, headings, tables, and equations mixed together with no explicit labeling of what is a definition, what is a result, and what is a claim. Even after that content is cleaned up, one paper is still a tiny amount of data. Shifting a base model's weights in a stable, generalizable way normally takes far more examples than a single document can supply. Both problems have to be solved before fine-tuning is realistic: structure the content, then multiply it.

KEY POINTS

- A base model has no knowledge of a document it was not trained on and cannot answer questions about it reliably from parametric memory alone.
- Fine-tuning requires structured input-output examples, not raw paragraphs of text.
- Even after structuring, a single source document rarely contains enough examples to fine-tune a model meaningfully.
- This creates a two-part problem: extract structure, then generate enough volume.

The quantum battery paper

A user wants to ask a chatbot "how do quantum batteries store energy?" and get an answer grounded in one specific paper. Without adaptation, the base model either hallucinates an answer from general training data or admits it doesn't know — because that paper's specific claims were never part of its weights.

02 Extracting structured context from raw documents

0:41

Tools like Docling turn PDFs and scans into structured data — sections, headings, tables — using OCR and parsing, which is the first step before any generation happens.

The first stage of the pipeline is extraction: converting a PDF, scanned document, or other raw format into structured data a program can reason about. Open source tools such as Docling do this by combining OCR (optical character recognition, which turns pixels into text) with document parsing that identifies structural elements — this text block is a heading, this one is a table cell, this one is a caption. The output is no longer a single blob of text; it is a table of concepts, definitions, and their relationships to each other.

Once content is structured, the next move is to teach the model how to respond, not just hand it more raw information. This is done by writing a small number of question-and-answer pairs that demonstrate the desired style and accuracy of an answer — asking a question a real user might ask and pairing it with the correct answer drawn from the structured content. This hand-written set is called seed data. It does not need to be large; its job is to be a clean, correct example of the target behavior, because everything generated afterward will be modeled on it.

KEY POINTS

- OCR converts scanned or image-based pages into machine-readable text.
- Parsing identifies structure — headings, sections, tables — that a flat text blob doesn't have.
- The extraction step's output is structured data: concepts and definitions, not paragraphs.
- Seed data is a small, hand-written set of question-and-answer pairs that exemplifies correct, on-topic answers.
- Seed data quality matters more than seed data quantity, because it becomes the template for generation.

From PDF to a Q&A seed pair

A Docling-style pipeline extracts a paragraph defining a quantum battery's charging mechanism. A human then writes a seed example pairing a natural question with the correct answer drawn from that paragraph.

```
{  
  "question": "How do quantum batteries store energy?",  
  "answer": "Quantum batteries store energy in the collective quantum state of coupled subsystems, exploiting entanglement to allow charging power to scale faster than in classical batteries.",  
  "source_section": "3.1 Charging Mechanisms"  
}
```

03 Generating synthetic data from seed examples

1:28

Once seed data exists, a generation model studies its patterns and produces new, similar question-and-answer pairs at scale — this is synthetic data generation.

With a small set of seed question-and-answer pairs in hand, the next stage uses a model to learn the distribution of that real data — the kinds of questions asked, the tone of the answers, the relationships between concepts — and generate new examples that follow the same pattern. For a document chatbot, this typically means turning a handful of manually written Q&A pairs into many more realistic ones covering more of the source material, without a human writing each one by hand.

Open source projects such as SDGHub package this as a synthetic data generation flow: a data pipeline with distinct stages that generates candidate examples, transforms them into the right format, and validates them before they're usable. Thinking of this as a pipeline rather than a single generation call matters, because raw model output is not automatically trustworthy — it needs the transform and validate stages downstream to catch mistakes before they end up in a training set.

KEY POINTS

- Synthetic data generation studies the pattern in seed data and produces new examples following that same pattern.
- For Q&A chatbots, this expands a few manually written examples into a much larger realistic set.
- SDGHub is an open source project that implements this as a multi-stage flow: generate, transform, validate.
- Treating generation as a pipeline (not one call) is what makes the output safe to add to a training set.

Expanding one seed pair into several synthetic ones

Starting from the single seed pair about quantum battery charging, a generation flow produces new Q&A pairs that probe related but distinct facts from the same source section, in the same question style as the seed.

```
// seed
{"question": "How do quantum batteries store energy?", "answer": "...entanglement..."}

// synthetic outputs modeled on the seed's style
{"question": "What role does entanglement play in quantum battery charging?", "answer": "..."}
{"question": "Why can quantum batteries charge faster than classical ones?", "answer": "..."}
{"question": "What happens to charging power as the number of coupled subsystems increases?", "answer": "..."}

```

04 Validating synthetic data before you trust it

2:14

Generated examples are checked for faithfulness, relevance, and diversity, and the whole flow can run locally against your own model endpoint so source data never leaves your environment.

A generated question-and-answer pair is only useful if it's actually correct and useful, so the validate stage runs automated checks on every candidate. Faithfulness asks whether the answer is actually supported by the source material, not invented. Relevance asks whether the answer actually addresses the question that was asked. Diversity asks whether the dataset as a whole covers a broad enough range of the source material, rather than clustering around the same few facts the seed data happened to touch.

These flows can run entirely locally, connecting to any OpenAI-API-compatible endpoint — including a self-hosted model served through Ollama or vLLM, or a hosted model such as Claude — to do the generation, critique, and faithfulness scoring. Running it this way means source data never has to leave your environment, which matters when the source documents are sensitive. Once a batch of synthetic data passes validation, it can be exported as CSV or JSON and dropped straight into a model training or fine-tuning pipeline.

KEY POINTS

- Faithfulness checks whether a generated answer is actually grounded in the source document.
- Relevance checks whether the generated answer actually answers the generated question.
- Diversity checks whether the dataset spans the source material broadly, not just a narrow slice.
- The generation and validation flow can connect to any OpenAI-compatible endpoint, including locally hosted models (Ollama, vLLM) or Claude.
- Running locally keeps source data inside your own environment instead of sending it to a third party.
- Validated data exports as CSV or JSON, ready to feed into a training pipeline.

A faithfulness check catching a bad generation

A generated pair claims quantum batteries "never lose charge over time." The source paper never makes that claim, so a faithfulness checker — itself a model prompted to compare the answer against the retrieved source passage — flags the pair and it's discarded rather than added to the training set.

```
prompt = f"""
Source passage:
{source_passage}

Generated answer:
{answer}

Is every claim in the generated answer supported by the source passage?
Reply FAITHFUL or UNFAITHFUL, and quote the unsupported claim if any.
"""

# UNFAITHFUL -> pair is dropped before export
```

05 Why control over the data matters

2:55

Synthetic data generation isn't just about producing more text — it gives you privacy preservation, class balancing, and a reproducible pipeline you can run before every deployment.

Because synthetic examples are generated rather than collected, you get properties that raw real-world data usually can't give you. You can preserve privacy by generating data that is statistically similar to your real source — same patterns, same distribution — but contains no real identifiers, since nothing is copied verbatim from a private record. You can balance rare classes: if a category of question or outcome is underrepresented in your real data, you can generate more examples of it specifically, rather than waiting to collect more real occurrences. You can augment limited domains where you simply don't have much source material to begin with, and you can generate test data to exercise a pipeline before it ever touches production.

The generate-validate-deploy sequence being made of repeatable, automatable steps is what makes it reproducible: run the same pipeline against the same seed data and configuration, and you get a dataset with the same statistical properties again. That reproducibility is what makes the approach usable in enterprise settings, where a training set needs to be auditable and regenerable, not a one-off artifact nobody can recreate. Whether the goal is fine-tuning a small domain-specific model or building an agent that answers questions from technical papers, synthetic data generation is what makes reaching production scale practical.

KEY POINTS

- Synthetic data can match the statistical distribution of real data without containing real identifiers, which supports privacy preservation.
- Underrepresented classes can be specifically targeted and expanded through generation, rather than waiting on real-world collection.
- Synthetic data can augment domains where source material is inherently limited.
- The generate-validate-deploy pipeline is repeatable, which makes the resulting dataset reproducible.
- Reproducibility is a practical requirement for enterprise AI workflows, where datasets need to be auditable.

Balancing a rare class

A support-ticket classifier's training set has thousands of "billing question" examples but only a dozen "security incident" examples. Rather than waiting for more real security tickets, a synthetic data flow generates additional security-incident-style Q&A pairs, seeded from the dozen real ones, until the class is adequately represented, then validates them for faithfulness before adding them to training.

Glossary

Seed data

A small, hand-written set of example question-and-answer pairs that demonstrates the correct pattern and style for a model to imitate when generating more examples.

Synthetic data

New data examples produced by a model rather than collected from the real world, generated to follow the statistical pattern of a smaller real dataset.

OCR (optical character recognition)

Software that converts images of text, such as a scanned page, into machine-readable text characters.

Faithfulness

A validation check on whether a generated answer's claims are actually supported by the source document, as opposed to invented.

Docling

An open source tool that converts PDFs and scanned documents into structured data — sections, headings, tables — using OCR and parsing.

SDGHub

An open source project that runs synthetic data generation flows: pipelines that generate, transform, and validate synthetic examples.

OpenAI-compatible endpoint

An API that accepts requests in the same format as OpenAI's API, letting tools switch between providers (self-hosted, Claude, etc.) without changing code.

Fine-tuning

Continuing to train an existing base model on a smaller, task-specific dataset so its weights shift toward that task.

Check yourself

Answer first, then open each one.

Why can't you fine-tune a model directly on a raw research paper's PDF text?

Fine-tuning is supervised learning: it needs structured input-output examples, and a raw PDF is unstructured prose mixed with tables and equations with no labeled inputs or outputs. It also usually isn't enough volume by itself to shift a base model's weights.

Why does the generation stage need seed data before it can produce synthetic examples?

The generation model works by learning the distribution and pattern of real examples and imitating it. Without seed data there's no pattern to learn from, so the model would have no basis for what a correct, on-topic answer looks like.

Why is synthetic data generation described as a pipeline rather than a single generation step?

Raw model output isn't automatically trustworthy. A pipeline separates generation from transformation and validation, so faithfulness, relevance, and diversity can be checked and bad examples discarded before anything reaches a training set.

How can synthetic data preserve privacy when the source data contains sensitive information?

Because synthetic examples are generated to match the statistical pattern of the source rather than copied from it, they can carry the same structure and distribution without containing any of the source's real identifiers.

Why does running the generation flow locally against a self-hosted endpoint matter for sensitive source documents?

Local execution means the source data is only ever sent to a model endpoint inside your own environment, so it never has to be transmitted to an external third-party service to generate synthetic examples.

Why is reproducibility described as essential for enterprise AI workflows?

An enterprise training set needs to be auditable and regenerable rather than a one-off artifact. Because the generate-validate-deploy pipeline is made of repeatable steps, running it again on the same seed data and config reproduces a dataset with the same properties, which supports auditing and iteration.

Where to go next

- Try Docling on a PDF you have and inspect the structured output it produces before writing any seed data.
- Look at SDGHub's example flows to see how generate/transform/validate stages are actually configured.
- Write 5-10 seed Q&A pairs from a document of your choosing and compare a generated batch against them for faithfulness by hand.
- Read up on OpenAI-compatible endpoints and try pointing a generation flow at a local Ollama or vLLM model instead of a hosted API.
- Investigate class-imbalance techniques in classical ML and compare them to synthetic data augmentation for the same problem.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.